

TEMA 3

INTRODUCCIÓN A P.O.O: DESARROLLO DE CLASES

2ª PARTE

PROGRAMACIÓN

JAKELIN SAYEL



“ El automóvil, el televisor, el vídeo, la computadora personal, el teléfono celular y demás contraseñas de la felicidad, máquinas nacidas para «ganar tiempo» o para «pasar el tiempo», se apoderan del tiempo”



Eduardo Galeano

ÍNDICE

Principios básicos de la POO.

1. Sobrecarga de métodos
2. Paso de parámetros. Paso por valor y paso por referencia
3. Ámbito de atributos y variables
4. Método ToString.Override
5. Librerías y paquetes de clases
6. Introducción a HERENCIA
7. Recursividad
8. Clases Anidadas
9. Argumentos de longitud variable
10. Argumentos del método MAIN



PRINCIPIOS BÁSICOS DE LA ORIENTACIÓN A OBJETOS

→ **Abstracción:** Definición de las características de los objetos y de sus diferencias con los demás. Se centra en qué hace un objeto más que en cómo lo hace. Nos encontramos con la importancia de una buena definición de la interfaz de una clase que nos permita modificar el cómo sin cambiar el qué. La abstracción se puede interpretar como qué hace una clase vista desde fuera.



→ **Encapsulamiento:** Se refiere a la ocultación del estado de un objeto a los demás.

Los atributos que representan el estado del objeto sólo serán accesibles mediante operaciones que presente el propio objeto. No necesariamente todos los elementos del estado del objeto deberán poder consultarse y/o modificarse.



Ambos se complementan y están relacionados: Al encapsular ocultamos los detalles menos importantes y mostramos los esenciales. lo que es esencial se hace por el proceso de abstracción.

PRINCIPIOS BÁSICOS DE LA ORIENTACIÓN A OBJETOS

- **Modularidad:** Permite dividir el programa objetivo en diferentes partes más pequeñas e independientes entre sí. Idealmente cada parte (por ejemplo una clase) será completamente independiente de otra pero no siempre es posible. Es necesario tener en cuenta a la hora de diseñar la división en módulos de nuestro programa aumentar la independencia y mantener la cohesión interna de las clases.

La cohesión es la “cercanía” conceptual de los componentes de una clase.

Ejemplo: Clase Jugador y Clase Arma

- Un jugador puede equipar un arma

```
Jugador jug= new Jugador();
```

```
Arma arma = new Arma();
```

```
arma.setDaño(10);
```

```
jug.equipa(a);
```

```
arma.recargaVida(); MAL ← MAL
```

Mal diseño:

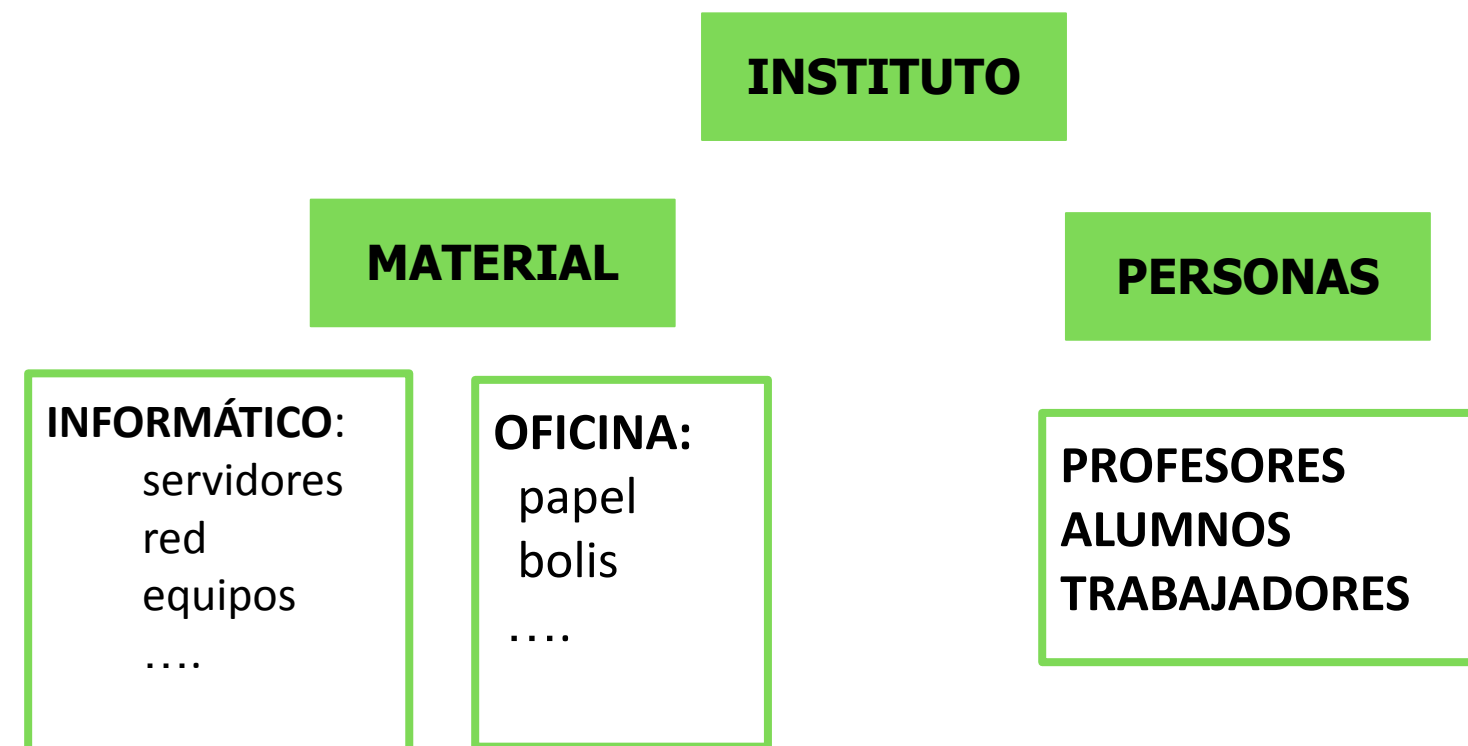
- El método de la clase Arma recarga vida no está cohesionado con las cosas que tiene que hacer el arma.
- Arma también depende de jugador de una forma artificial

PRINCIPIOS BÁSICOS DE LA ORIENTACIÓN A OBJETOS

- **Jerarquía:** Permite establecer una organización de las clases. En Programación Orientada a Objetos los dos tipos más importantes de jerarquía son:
- La especialización: (Es un) Un coche es un vehículo y por lo tanto tiene las características de un vehículo y las concreta.
 - La composición: (Es parte de) las ruedas forman parte de un vehículo.

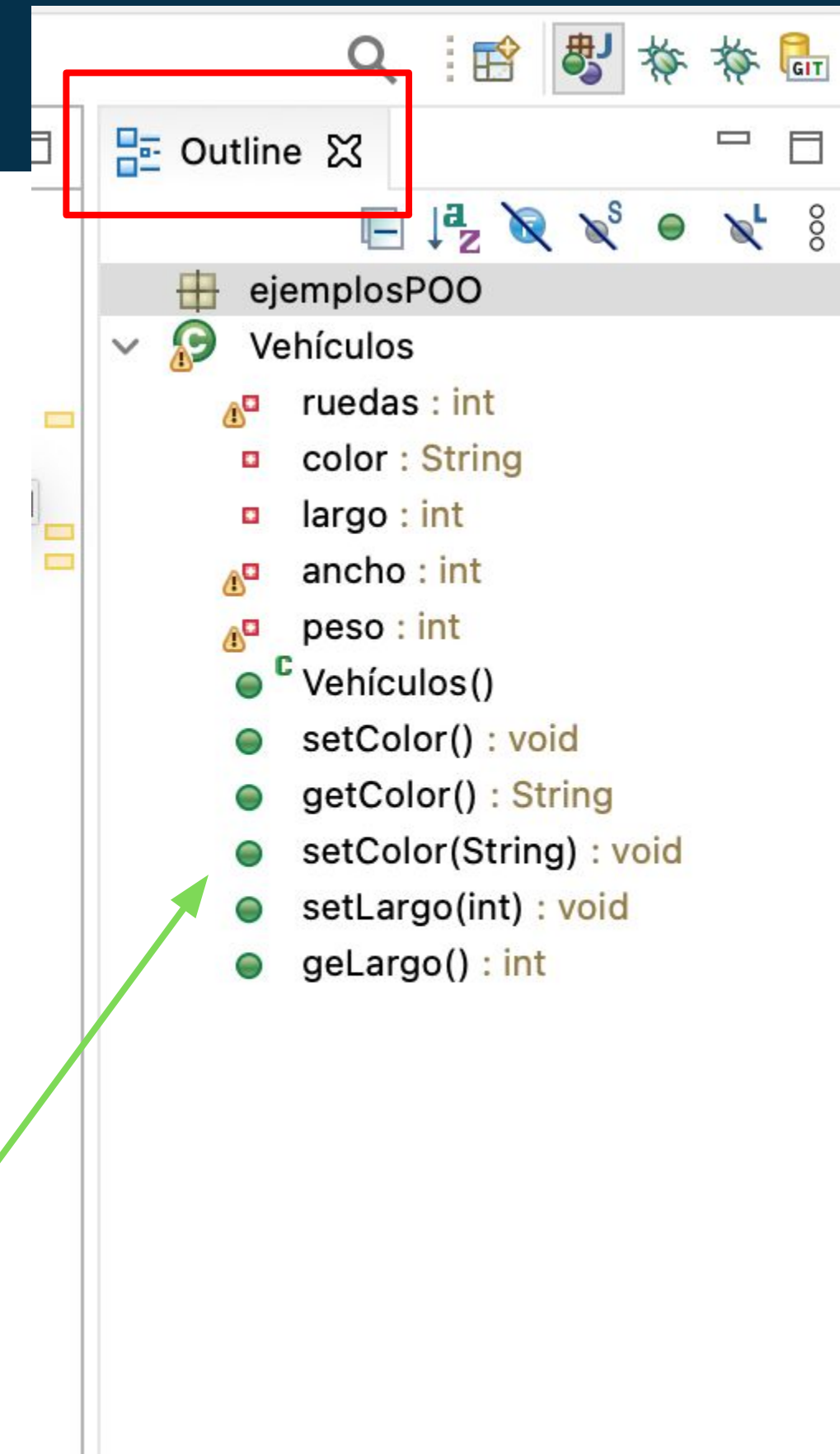
LO VEREMOS MÁS ADELANTE

Ejemplo:



1. SOBRECARGA DE MÉTODOS

- Una de las ventajas que ofrece la programación orientada a objetos es poder tener en una misma clase varios métodos con el mismo nombre, a esto se le llama sobrecarga de métodos.
- La gran ventaja es que, si tenemos varios métodos que van a realizar la misma operación no necesitamos asignarle distinto nombre, que nos hace más difícil aprender los nombres, sino que podemos llamarlos igual a todos ellos.
- Para que un método pueda sobrecargarse es imprescindible que se dé la siguiente condición: **cada versión del método debe distinguirse de las otras en el número o tipo de parámetros**. El tipo de devolución puede ser o no el mismo.
- Vamos a ver cómo lo haríamos. Pero antes, veamos una nueva pestaña de Eclipse: **"Outline"** , sino la tenéis visible -> Window->Show View->Outline
- Nos podemos desplazar a un método o propiedad pinchando sobre el nombre.

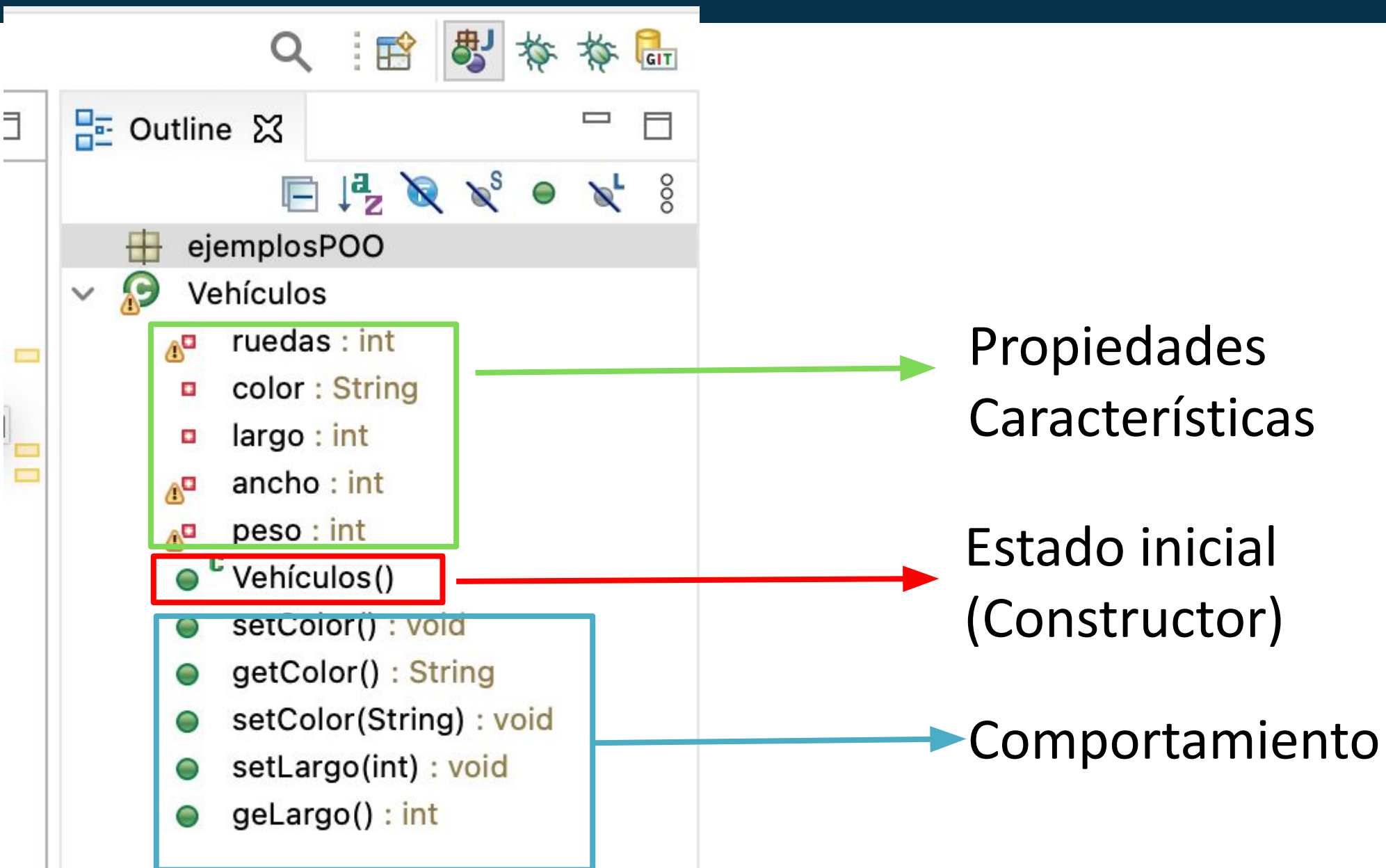


1. SOBRECARGA DE MÉTODOS

- Recordamos el ejemplo vehículo de la parte 1 del tema, que tenía varios atributos como color, tamaño, ruedas...etc. Que eran comunes a todos los objetos.
- Pero imaginad que cuando nosotros queremos comprar un coche, puede ser que queramos algún extra, como por ejemplo gps, climatizador...
- Si los inicializamos en el constructor como hacemos con los atributos de ruedas, color y demás estamos obligando a que todos los coches tengan esas características.
- ¿Cómo lo solucionamos?

```
public class Vehículos {  
    private int ruedas;  
    private String color;  
    private int largo;  
    private int ancho;  
    private int peso;  
  
    public Vehículos() {  
        ruedas=4;  
        color="sin color";  
        largo=3;  
        ancho=2;  
        peso=1;  
    }  
}
```


1. SOBRECARGA DE MÉTODOS



- Si metemos en el constructor los "extras" obligamos a inicializarlos cuando, tal vez no tenga gps
- Es cierto que las podríamos inicializar en el constructor a false la variable gps (boolean gps), pero sería redundante porque el valor por defecto de una variable tipo boolean ya es false.
- Para solucionarlo existe la sobrecarga de métodos o constructores.
- **Se pueden crear varios métodos que se llamen igual pero que un N° diferente de argumentos o distinto tipo de argumento**

1. SOBRECARGA DE MÉTODOS



- La palabra reservada **this** tiene también otra utilidad.
- Si la primera sentencia de un constructor tiene la palabra **this**, entonces el constructor llama a otro constructor de la misma clase.

```
public VehiculosSob(int ruedas, int largo, int peso, double ancho)
{
    this.ruedas=ruedas;
    color="sin color";
    this.largo= largo;
    this.ancho=ancho;
    this.peso=peso;
}
public VehiculosSob(int ruedas) {
    //este this está llamando al constructor anterior
    this(ruedas,4,1500,2);
}
```

- En este ejemplo hay sobrecarga de constructores.
- Desde el segundo constructor (el que tiene un único parámetro) se llama utilizando la palabra reservada **this** al constructor.
- imaginad que tenemos los mismos valores para 100 vehículos que son camiones de los 500 que tenemos y lo único que cambia son las ruedas

1. SOBRECARGA DE MÉTODOS

→ Otro ejemplo de esto lo tenemos en la clase String, que nos proporciona el método `valueOf()`, al que se le pueden pasar distintos tipos de datos (int, double, boolean ..)

```
6 public static void main(String[] args)
7 {
8
9     String resultadoEntero, resultadoDouble, resultadoUnion;
10    int valorEntero = 10;
11    double valorDouble = 6.4;
12
13    // Al no ser del tipo donde van a ser asignados falla
14    // resultadoUnion = valorEntero+valorDouble;
15
16    // Hay que convertir a String los valores
17    resultadoEntero = String.valueOf(valorEntero);
18
19    resultadoDouble = String.valueOf(valorDouble);
20
21    resultadoUnion = resultadoEntero + resultadoDouble;
22
23    System.out.println("Resultado: " + resultadoUnion);
24
25 }
```


1. SOBRECARGA DE MÉTODOS

- Como vemos en el ejemplo, al método de la clase `String` `valueOf`, se le puede pasar un entero o un `double`, esto en realidad son dos métodos distintos que se llaman de la misma forma, pero que tienen distintos parámetros. Esto es la sobrecarga de métodos.
- La gran ventaja es que, si tenemos varios métodos que van a realizar la misma operación (por ejemplo convertir un tipo básico en un `String`), no necesitamos asignarle distinto nombre, que nos hace más difícil aprender los nombres, sino que podemos llamarlos igual a todos ellos.
- Para que un método pueda sobrecargarse es imprescindible que se dé la siguiente condición: **cada versión del método debe distinguirse de las otras en el número o tipo de parámetros. El tipo de devolución puede ser o no el mismo.**

Ejemplos de sobrecarga:

```
public void calculo(int k) {...}
```

```
public void calculo(String s) {...}
```

```
public void calculo(int k, boolean b) {...}
```

!!! IMPORTANT !!!



1. SOBRECARGA DE MÉTODOS

- En cambio, los siguientes métodos incumplen alguna de las condiciones de la sobrecarga, por tanto, no podrían estar en la misma clase:

```
public void calculo(int k) {...}
```

```
public int calculo(int k) {...} //cambia el tipo de devolución, pero la lista de devolución es igual al anterior
```

```
public void calculo(int n) {...} //este método es idéntico al primero, no importa el nombre del argumento, sino su tipo. Con lo que no podríamos usarlo
```

- La sobrecarga de métodos nos permite, por tanto, disponer de diferentes versiones de un método para llevar a cabo una determinada operación. A la hora de invocar a un método sobrecargado en un objeto, el compilador identificará la versión del método que se quiere invocar por los argumentos utilizados en la llamada.

1. SOBRECARGA DE MÉTODOS

Ejemplo:

Tenemos la siguiente clase con el método muestra sobrecargado

```
3 public class Ejemplo
4 {
5     public void muestra()
6     {
7         System.out.println("Texto de ejemplo.");
8     }
9
10    public void muestra(String s)
11    {
12        System.out.println("Texto que vale: " + s);
13    }
14
15 }
```

3

La salida será:

Texto de ejemplo.

Si ejecutamos la siguiente clase

```
3 public class UsoEjemplo
4 {
5     public static void main(String[] args)
6     {
7         Ejemplo ej = new Ejemplo();
8         ej.muestra();
9     }
10 }
```

2

Se ha ejecutado la versión sin parámetros del método muestra().

1. SOBRECARGA DE MÉTODOS

Ejemplo:

Sin embargo, si utilizamos la siguiente clase:

```
3 public class UsoEjemplo
4 {
5     public static void main(String[] args)
6     {
7         Ejemplo ej = new Ejemplo();
8         ej.muestra("Hola!!");
9     }
10 }
```

La salida será:

Texto que vale: Hola!!

Se ha ejecutado la versión con parámetros del método muestra().

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

- Como hemos visto hasta ahora, Java tiene dos tipos de variables: primitivas y objetos.
- En una variable primitiva guardamos directamente el valor que tiene la misma, sin embargo en una variable de tipo objeto, guardamos la referencia a la dirección de memoria donde está almacenado el valor.
- Técnicamente, se organiza de la siguiente forma: las variables primitivas siempre son almacenadas en la memoria stack, sin embargo en el caso de los objetos estos son almacenados en dos etapas, los objetos actuales son almacenados en la memoria heap y la referencia a objetos se encuentra almacenada en la memoria stack que apunta a los objetos.



2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

Por valor y Por referencia

Que significa "Por valor" y "Por referencia".

- **Por valor:** cuando los argumentos son pasados por valor a los métodos, significa que se realiza una copia de la variable y esta es enviada al método y no la original, entonces todos los cambios realizados dentro del método sólo afectan a la copia actual.
- **Por referencia:** cuando los argumentos son pasados por referencia, significa que la referencia o el puntero a la variable original son pasadas a los métodos y no el dato original



2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

¿Como son pasados los argumentos en Java?

En Java, los argumentos son siempre pasados por valor independientemente del tipo de variable. Cada que el método es invocado, pasa lo siguiente:

- Una copia del argumento es pasado en la memoria stack y la copia es pasada en el método
 - Si la variable original es primitiva, es simple, una copia de la variable es creada en el stack y esta se pasa al método
 - Si la variable original no es primitiva, entonces una nueva referencia o apuntador es creado dentro de la memoria stack que apunta al objeto actual y la nueva referencia es pasada al método, (en esta etapa, 2 referencias apuntan al mismo objeto).

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

EJEMPLOS:

En el siguiente ejemplo, tratamos de validar “en java siempre se realiza el pase por valor” pasando un conjunto de tipos de argumentos.

Paso de argumentos primitivos:

```
6 public static void main(String[] args)
7 {
8     int x = 1;
9     int y = 2;
10
11     System.out.println("Valor antes de modificar de x: " + x + ", y: " + y);
12     modificarValores(x, y);
13     System.out.println("Valor despues de modificar de x: " + x + ", y: " + y);
14 }
15
16 public static void modificarValores(int x, int y)
17 {
18     x = 100;
19     y = 200;
20     System.out.println("Valor dentro del método modificar de x: " + x + ", y: " + y);
21 }
```

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

EJEMPLOS:

Paso de argumentos primitivos:

Salida

```
Valor antes de modificar de x: 1, y: 2  
Valor dentro del método modificar de x: 100, y: 200  
Valor despues de modificar de x: 1, y: 2
```

Las dos variables **x** e **y** son tipos primitivos y son almacenadas en la memoria stack. Cuando se llama a `modificarValores()`, se realizan dos copias dentro de la memoria stack y luego estas son pasadas al método. Las variables originales no fueron enviadas al método y cualquier modificación dentro del método sólo afectan a las copias

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

EJEMPLOS:

Paso de argumentos de tipo objeto:

```
6 public static void main(String[] args)
7 {
8     Estudiante juan = new Estudiante();
9
10    System.out.println("Nombre del estudiante antes de asignar: " + juan.getNombre());
11    juan.setNombre("Juan");
12    System.out.println("Nombre del estudiante despues de asignar: " + juan.getNombre());
13    modificaEstudiante(juan);
14    System.out.println("Nombre del estudiante despues del método de modificar: " + juan.getNombre());
15 }
16
17 public static void modificaEstudiante(Estudiante juan)
18 {
19     juan.setNombre("Ana");
20 }
```

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

EJEMPLOS:

Paso de argumentos de tipo objeto:

Salida	Nombre del estudiante antes de asignar: null
	Nombre del estudiante despues de asignar: Juan
	Nombre del estudiante despues del método de modificar: Ana

El objeto juan es creado dentro de la memoria heap y la referencia se encuentra en la memoria stack, cuando se llama a ***modificaEstudiante()***, una copia de la referencia es creado dentro del stack y esta se pasa al método. Cualquier modificación dentro de los atributos del objetos dentro del método son reflejadas en la referencia original.

2. PASO DE PARÁMETROS: POR VALOR Y REFERENCIA

CONCLUSIÓN

En Java, los argumentos son pasados por valor, la copia puede ser una referencia o una variable dependiendo del tipo original de la variable. Utilizar las siguientes recomendaciones para entender como la modificación de argumentos dentro del método afectan a la variable original:

1. La modificación de valores tipos primitivos, nunca afectan a la variable original
2. Si se cambia la referencia al objeto dentro del método esto nunca afecta a la referencia original, sin embargo esto crea un nuevo objeto en la memoria heap.
3. Modificar los atributos de un objeto pasado como argumento se refleja fuera del método.



3. ÁMBITO DE ATRIBUTOS O VARIABLES

→ El ámbito de una variable define su alcance de uso, o lo que es lo mismo, en que secciones de código una variable estará disponible. Fuera de este ámbito, una variable no podrá ser accedida (no existe).

Existen varios tipos:

- **A nivel global de clase (atributo):** cuando declaramos una variable al inicio de la clase, esta variable se podrá leer y modificar en cualquier parte del programa. Estas variables se pueden utilizar sin haber sido inicializadas de manera explícita, ya que se inicializan cuando se crea un objeto de la clase (con el constructor, ya sea el de defecto o uno creado).
- **Variable local (a nivel de método):** Son variables que se declaran dentro de un método, su ámbito de utilización está restringido al interior del método y no admiten ningún tipo de modificador (public, private ..). Una variable local se crea en el momento en que se hace la llamada al método, destruyéndose cuando finaliza la ejecución de éste (se dice que la variable sale del ámbito). Una variable local también puede estar declarada en el interior de un bloque de código (ej un if) sólo que, en este caso, su uso está restringido al interior de ese bloque. Toda variable local tiene que ser inicializada explícitamente antes de ser utilizada.

4.Método toString

- Lo que hace el toString es que podamos poner cualquier información como si fuese un String, es un método de la Clase objeto que es la clase más usada en java lo veremos más adelante.
- @override es obligatorio cuando se usa toString, simplemente se usa para no sobrescribir ,lo veremos en herencia.
- El método **toString** nos permite mostrar la información completa de un objeto.

```
@Override
```

```
public String toString() {
```

```
    String ret = this.dia + "/" + this.mes + "/" + this.anno;
```

```
    return ret;
```

```
}
```

```
// los dos métodos hacen lo mismo devolver los datos de fecha
```

```
String devolverFecha() {
```

```
    String ret = this.dia + "/" + this.mes + "/" + this.anno;
```

```
    return ret;
```

```
}
```

5. LIBRERÍAS Y PAQUETES DE CLASES

- En Java y en varios lenguajes de programación más, existe el concepto de librerías.
- Una librería en Java se puede entender **como un conjunto de clases, que poseen una serie de métodos y atributos.**
- Lo realmente interesante de estas librerías para Java es que **facilitan muchas operaciones.** De una forma más completa, las librerías en Java nos permiten **reutilizar código**, es decir que podemos hacer uso de los métodos, clases y atributos que componen la librería evitando así tener que implementar nosotros mismos esas funcionalidades.
- Imaginemos cómo hacer un código para conseguir que nuestra aplicación imprima algo por pantalla. Por medio de librerías, para imprimir texto en pantalla en Java basta con usar `System.out.print()`, en Java, esto es así de fácil y todo es gracias a las librerías, la clase `System` que pertenece a la librería `java.lang` (ésta es la librería estándar de Java y no es necesario importarla). Cabe mencionar que estas librerías no sólo poseen estas clases sino que poseen una gran cantidad de clases Java adicionales que nos permiten llevar a cabo operaciones muy complejas con gran facilidad.

5. LIBRERÍAS Y PAQUETES DE CLASES

- Lo más interesante de las librerías en Java, es que nosotros también podemos crearlas y hacer uso de ellas al interior de nuestros proyectos. Básicamente un paquete en Java puede ser una librería, sin embargo una librería Java completa puede estar conformada por muchos paquetes **más**. Al importar un paquete podemos hacer uso de las clases, métodos y atributos que lo conforman, así que eso de las librerías no es solo trabajo de otros sino que podemos crear nuestras propias librerías y usarlas, incluso podemos hacer uso de un proyecto completo nuestro al interior de otro.

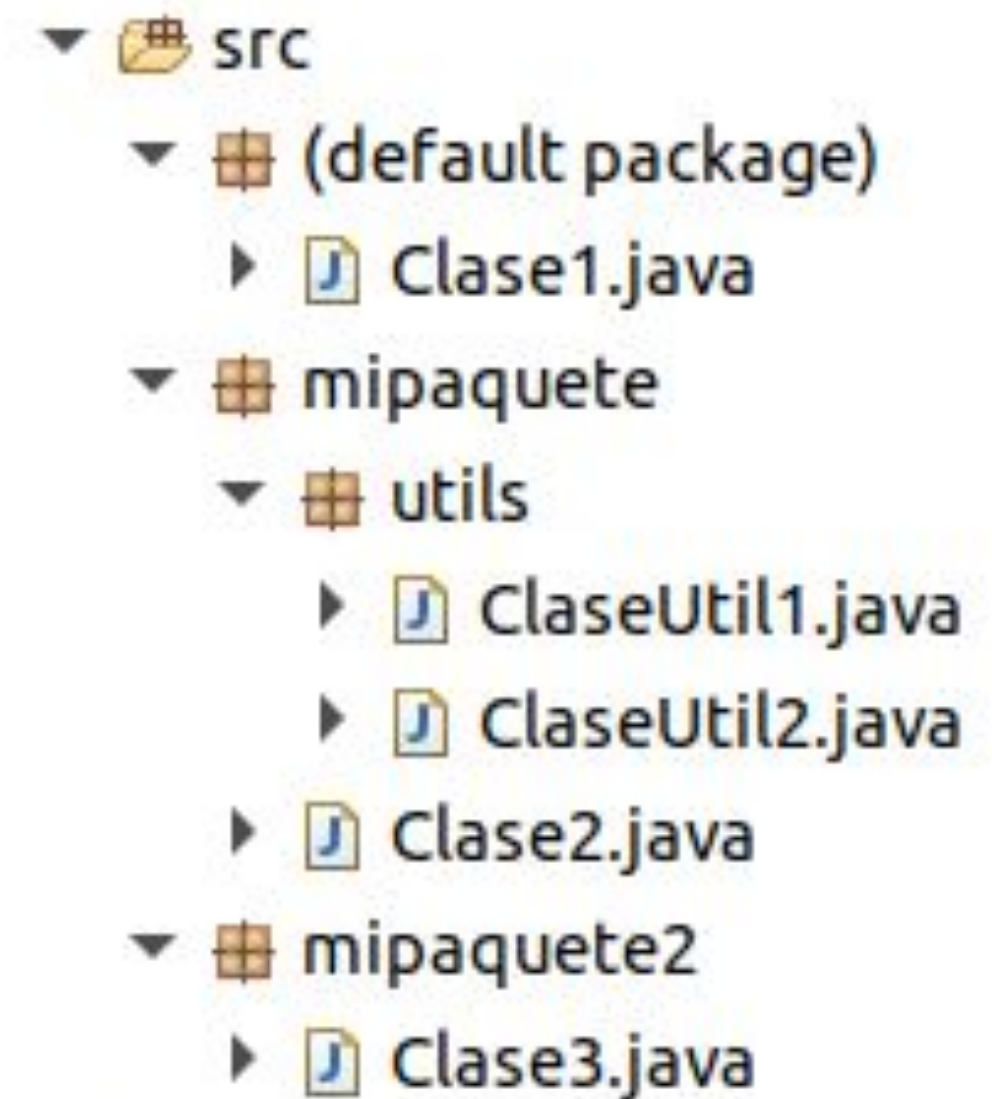
IMPORT JAVA (Importar librerías en Java)

- Hacer uso de librerías tanto propias como externas, o librerías propias de Java, es bastante fácil. Básicamente lo único que debemos saber es que para importar librerías en Java se usa la palabra clave **import seguido de la "ruta" del paquete o clase que deseamos agregar al proyecto**. Cabe resaltar que el import permite agregar a nuestro proyecto una o varias clases (paquete) según lo necesitemos.

5. LIBRERÍAS Y PAQUETES DE CLASES

IMPORT JAVA (Importar librerías en Java)

- Para explicar cómo importar librerías partiremos del siguiente ejemplo:
- En este ejemplo, vemos que hay 5 clases, la Clase1, se crea en el paquete por defecto, que es como si no hubiese paquete. La Clase2 se crea dentro de mipaquete, y la Clase3 dentro de mipaquete2. Y las clases ClaseUtil1 y 2, se han creado dentro del paquete mipaquete.utils
- Ya sabemos la estructura de nuestros paquetes y necesitamos importarlos al interior de alguna otra clase para hacer uso de ellas, para esto, Java nos proporciona el comando import.



5. LIBRERÍAS Y PAQUETES DE CLASES

Jerarquía de paquetes

- Los paquetes se organizan de forma similar al escritorio de nuestro ordenador, en donde:
 - Las clases serían los archivos.
 - Cada paquete sería como una carpeta.
 - Cada paquete puede además contener otros paquetes, **paquetes anidados**.
 - Para hacer referencia a una clase dentro de una estructura de paquetes, habrá que indicar la trayectoria completa desde el paquete raíz de la jerarquía hasta el paquete en el que se encuentra la clase, indicando por último el nombre de la clase (**path absoluto**).
 - Para indicar al paquete que pertenece, se utiliza la palabra reservada **package**.
 - Este mecanismo, nos *evita conflictos de nombres de clases* y, así, localizarla fácilmente dentro de una estructura.



5. LIBRERÍAS Y PAQUETES DE CLASES



Concepto de paquete

- La **encapsulación** de la información es llevada a cabo dentro de las clases, permitiendo realizar el proceso de ocultación de la información.
- El proceso semejante que afecta a las clases en sí se denomina **empaquetado**. Este proceso es un nivel superior de encapsulamiento y ocultación de la información.
- Por lo tanto, un **paquete** consiste en un conjunto de clases relacionadas entre sí y agrupadas bajo un mismo nombre. Normalmente se encuentran en un mismo paquete todas aquellas clases que forman una biblioteca o que reúnen algún tipo de característica en común.



6. LIBRERÍAS Y PAQUETES DE CLASES



Concepto de paquete

- PAQUETE o PACKAGE es un conjunto de clases e interfaces que facilitan la gestión del acceso a las clases.
- Podemos decir que un paquete es una biblioteca de clases.
- El uso de Paquetes permite:
 - ◆ proteger clases restringiendo el acceso
 - ◆ definir un espacio de nombres para organizar y localizar las clases
- Si en una clase queremos usar clases de otros paquetes, debemos importar este último.



import paquete.subpaquete.subsubapquete....clase



5. LIBRERÍAS Y PAQUETES DE CLASES

Concepto de paquete

- Podremos incorporar todo el paquete o solo la clase que necesitamos usar.

```
import java.util.*;
```

```
import java.util.Date;
```

- Java, importa por defecto dos paquetes:

- 1.** Java.lang

- 2.** El paquete por defecto.



5. LIBRERÍAS Y PAQUETES DE CLASES

- Al igual que sucede con la declaración de los paquetes, realizar el import en Java también tiene una localización específica al interior del archivo java. La importación de librerías y demás siempre debe ir después de la declaración del paquete, si la clase no posee alguna declaración de paquete (pertenece al paquete por defecto), entonces los import deberán estar en las primeras líneas del archivo.
- Ejemplos para importar paquetes o clases en Java.

```
package mipaquete2;  
  
import mipaquete.utils.ClaseUtil1;  
  
public class Clase3  
{
```

- En el código básicamente estamos diciendo que la clase contenida en el archivo Java, estará en el paquete "mipaquete2", es decir, esa será la ruta de acceso a esa clase en el proyecto. En la siguiente línea hemos realizado el import de un paquete. Hemos indicado la ruta donde se encuentra la clase de queremos importar "mipaquete.utils" y el nombre de la clase "ClaseUtil1".

5. LIBRERÍAS Y PAQUETES DE CLASES

```
package mipaquete2;  
  
import mipaquete.utils.*;  
  
public class Clase3  
{
```

- En el código básicamente estamos diciendo que la clase contenida en el archivo Java, estará en el paquete "mipaquete2", es decir, esa será la ruta de acceso a esa clase en el proyecto. En la siguiente línea hemos realizado el import de un paquete. Hemos indicado la ruta donde se encuentra la clase de queremos importar "mipaquete.utils", y por medio del "*" hemos indicado a Java que queremos importar todas las clases pertenecientes a dicho paquete, puede ser una o más.

5. LIBRERÍAS Y PAQUETES DE CLASES

→ En este código la diferencia es que importamos un paquete que contiene otros paquetes, “mipaquete” contiene el paquete “utils”, y si tuviese más también estaríamos importando todas las clases contenidas dentro de cualquiera de los subpaquetes.

```
package mipaquete2;  
  
import mipaquete.*;  
  
public class Clase3  
{
```

- Hasta ahora hemos visto como importar nuestras propias clases o paquetes, para hacerlo con clases que nos proporciona Java, ya lo hemos hecho antes, por ejemplo para la clase Scanner.
- Si vamos a necesitar solo una clase, es recomendable importar solo esa clase concreta, sin embargo si vamos a utilizar varias clases de un paquete podemos importar el paquete y con un * importar todas las clases que contiene el mismo.

6. UTILIZACIÓN DE CLASES HEREDADAS

- **Herencia:** Una clase deriva de otra y adquiere todas sus características para luego ampliarlas o concretarlas. Por ejemplo una clase Empleado y otra clase Cliente podrían heredar de una clase Persona que contiene lo común a ambas (nombre, dirección, etc.).
- Luego un cliente podría tener por ejemplo unos datos de facturación y un empleado una nómina. También se conoce como especialización.
- **La herencia es básicamente una jerarquía** que implica que las clases inferiores mantienen todo lo de las superiores y añaden (pueden añadir) funcionalidad. Se dice que las clases inferiores heredan lo que tiene la superior. Si vamos a String y a Math podemos ver que ambas heredan (realmente todas heredan directa o indirectamente) de Object. Por ello aunque no se especifique tienen los métodos que ésta implementa.

6. UTILIZACIÓN DE CLASES HEREDADAS

- Los métodos que incluye **Object** realmente están para que se indique que todas las clases lo tendrán, pero se suelen sobrescribir (como veremos después de la herencia).
- Esto nos permite procesar Objetos de diferentes tipos como si fueran todos del elemento común que tengan en la jerarquía. Por ejemplo podríamos crear tanto un String como un Integer como si fuesen Object.
- Esto es lo que hace el recolector de basura cuando llama a finalize. Tiene una colección de objetos y como todos tienen que ser forzosamente de tipo Object (aunque tengan un tipo más específico) tendrán finalize seguro.

7. RECURSIVIDAD

- En Java un método puede invocarse a sí mismo.
- A este proceso se le llama recursividad o recursión, y al método que lo realiza se le denomina recursivo.

Todo método recursivo, debe contener:

- Caso base: Condición de salida mediante la cual se asegura que no se producen llamadas recursivas. Siempre tiene que existir un caso base, como mínimo.
- Llamada recursiva: Se tienen que realizar llamadas a sí mismo (recursiva), problemas más pequeños.

○ EJEMPLO

```
class Factorial {  
    int fact(int n) {  
        if (n==0) return 1;  
        return n*fact(n-1);  
    }  
}
```



7. RECURSIVIDAD

→ A la hora de solucionar un problema mediante una versión recursiva o iterativa, tenemos que tener en cuenta que las soluciones recursivas son menos eficientes que las iterativas debido a su carga computacional (tiempo de CPU y espacio en memoria), debido a las llamadas recursivas.

Dado el punto anterior, planteamos una solución recursiva cuando:

- Cuando tengamos una solución iterativa obvia al problema en cuestión, evitaremos resolverlo mediante recursividad (complejidad de la solución).
- En problemas iterativos complejos que, por su naturaleza, la solución recursiva es más fácil de implementar, utilizaremos recursividad. Esto conlleva una alta legibilidad del código resultante.
- Puede darse el caso que la solución recursiva resuelva el problema en repetidas ocasiones (redundancia), siendo así no indicada para solventar el problema

8. CLASES ANIDADAS

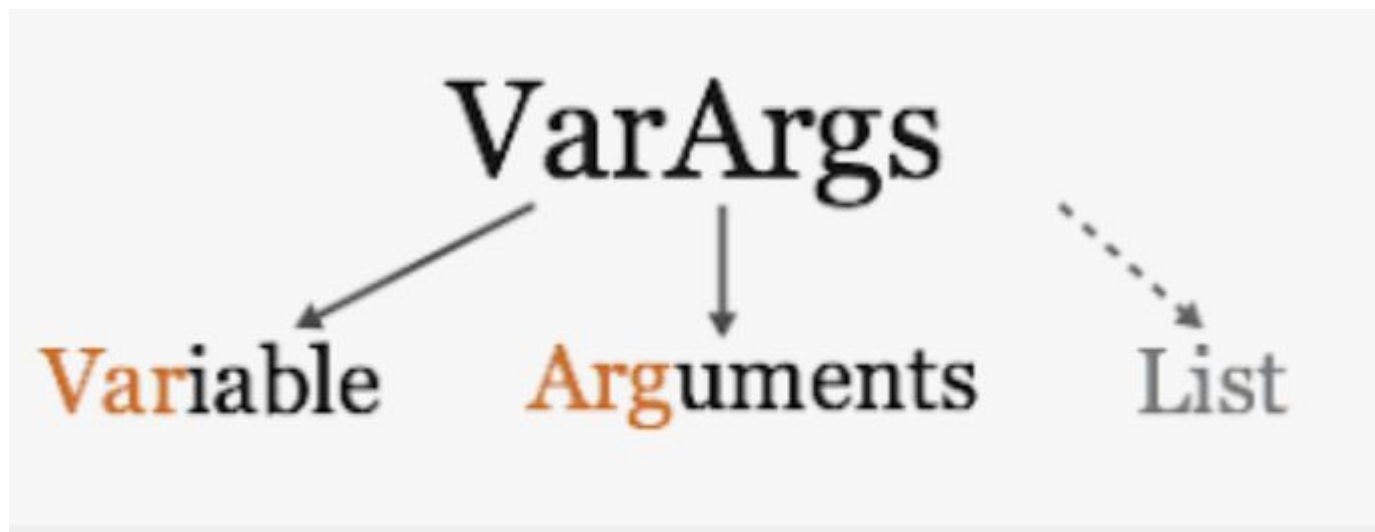
- En Java se pueden definir clases anidadas, es decir una clase que se define dentro de otra.
- A la clase anidada debe ser estática y es un miembro de la clase contenedora.
- La clase anidada tiene acceso a todas las variables y métodos de la clase externa, pero no al revés.
- En ocasiones se usan clases internas para ofrecer una serie de servicios que sólo necesitan usarse en su clase contenedora e incrementar el nivel de encapsulación u ocultamiento.

8. CLASES ANIDADAS

```
public class Composicion {  
    // Programa Java para demostrar el acceso a una clase anidada estática  
  
    //Declaración de variables  
    static int externo_x=10;  
    int externo_y=20;  
    private static int externo_privado=30;  
    //System.out.println("variable de la clase anidada: "+internaX);  
  
    //Declaracion Clase anidada  
    public static class ClaseAnidada{  
        int interna;  
  
        public ClaseAnidada() {}  
  
        void mostrar(){  
            System.out.println("externo_x: "+externo_x);  
            System.out.println("externo_privado: "+externo_privado);  
        }  
    }  
}  
  
public class ClaseAnidadaStaticDemo {  
    public static void main(String[] args) {  
        //Accediendo a la clase anidada estática  
        Composicion.ClaseAnidada objetoAnidado = new Composicion.ClaseAnidada();  
        objetoAnidado.mostrar();  
    }  
}
```

9. ARGUMENTOS DE LONGITUD VARIABLE

- Un método que **acepta un número variable de argumentos** se denomina método varargs.
- **La lista de parámetros de un método varargs no es fija**, sino de longitud variable.
- Se especifica:
static void vaTest(**tipo_arg ... nombre**)
que crea un vector nombre con elementos de tipo tipo_arg
- Puedo mezclar argumentos normales y de longitud variable. Este irá siempre al final.
Ejemplos : ArgsLongVar y ArgsLongVar1



Java varargs

9. ARGUMENTOS DE LONGITUD VARIABLE

```
// sencillo ejemplo de método con argumentos de longitud variable
```

```
import java.io.*;
import java.util.*;
```

```
class ArgsLongVar {
    static void vaTest(int ... v){
        System.out.println("Numero de argumentos: "+v.length);
        System.out.println("Contenido: ");
        for (int i=0; i<v.length;i++)
            System.out.println(" arg "+i+": "+v[i]);
    }
}
```

```
public static void main(String args[]){
```

```
    //Diferentes invocaciones de vaTest
    vaTest(10);           // 1 argumento
    vaTest(1, 2, 3);      // 3 argumentos
    vaTest();             //sin argumentos
}
```

```
// sencillo ejemplo de método con argumentos de longitud variable
```

```
import java.io.*;
import java.util.*;
```

```
class ArgsLongVar1 {
    static void vaTest(String...s){
        System.out.println("Numero de argumentos: "+s.length);
        System.out.println("Contenido: ");
        for (int i=0; i<s.length;i++)
            System.out.println(" arg "+i+": "+s[i]);
    }
}
```

```
public static void main(String args[]){
```

```
    //Diferentes invocaciones de vaTest
    vaTest("hola");        // 1 argumento
    vaTest("a", "b", "c"); // 3 argumentos
    vaTest();              //sin argumentos
}
```

```
}
```

10. ARGUMENTOS DEL MÉTODO MAIN

```
public static void main(String[] args)
```

- Al argumento podemos darle el nombre que queramos.
- Un programa java se puede ejecutar desde la línea de comandos del sistema operativo con la orden: **C:\> java nombrePrograma**
- Pero además, mediante esta orden, podemos enviar valores al programa.
Ejemplo: si tenemos un programa que se llama *ordenar* que ordena 5 números enteros, pero en lugar de leerlos por teclado los números se le pasan al programa como argumentos, este programa lo ejecutamos así:

```
C:\> java ordenar 4 6 3 7 1
```


10. ARGUMENTOS DEL MÉTODO MAIN

```
public class Saludo {  
  
    public static void main(String[] args) {  
        if (args.length > 1) { //si hay más de 1 parámetro  
            for(String arg:args) {  
                System.out.println("Hola "+arg);  
            }  
  
        } else if (args.length == 0) { //si no hay parámetros  
            System.out.println("Falta el nombre de la persona");  
        } else {  
            System.out.println("Buenos Días " + args[0]);  
        }  
    }  
}
```




¿Preguntas?